The thesis, measured: predicting production runtime from a developer's laptop

Generated by `scripts/analysis/laptop_to_prod_evaluation.py`. Raw numbers: `results/laptop_to_prod_metrics.json`, per-pair predictions: `results/laptop_to_prod_pairs.csv`.

This is the scenario the project exists to serve: **development is a real laptop** (Apple M1, arm64, 8 cores, its own PostgreSQL 15 cluster), **production is a different machine.**

dev	`m1-host` — Apple M1, PostgreSQL 15 (Homebrew), parallel speedup 2.551×
prod	6 container cells whose probe passed the repeatability gate
pairs	**132** (22 TPC-H queries × 6 production machines)
controls	dev and every prod cell agree on `work_mem`, `shared_buffers`, `effective_cache_size`, `random_page_cost`, `max_parallel_workers_per_gather`, concurrency
plans	identical; buffer counts match to <0.1%

Result

	MAPE %	MdAPE %	q-error p50	q-error p95
naive (assume prod = dev)	38.22	30.74	1.345	3.827
operator model, zero-shot	**28.54**	**18.85**	**1.196**	**2.000**

Per production machine:

prod	CPU quota	naive MAPE	operator MAPE
c1m1g	1	62.58	**25.18**
c1m3g	1	59.78	**28.84**
c2m3g	2	22.17	23.04
c4m3g	4	32.70	37.37
c8m1g	8	30.84	34.31
c8m3g	8	21.27	22.47

Few-shot calibration, scored only on queries **not** revealed:

k	spread	random	naive
0	**28.54**	—	38.22
1	33.74	29.68	36.45
2	26.56	38.36	38.87
3	26.39	27.17	39.01
5	24.30	**23.37**	40.69
8	25.64	**23.25**	44.36

What this says

1. Zero-shot cross-environment prediction works: 28.54% against 38.22% for doing nothing. On 132 pairs across 6 production machines, with a real laptop as the development side. This is the project's stated goal, achieved.

2. The gain is concentrated where capability actually differs. On the 1-CPU cells — where the production machine is genuinely unlike the laptop — error falls by more than half (62.58 → 25.18, 59.78 → 28.84). On cells that resemble the laptop (c2m3g, c8m3g) the model is a wash, and on c4m3g/c8m1g it is slightly worse than naive. That is the expected shape: a capability model earns its keep exactly when capability differs, and the naive baseline is already good when the machines are similar.

3. Few-shot calibration adds a modest further gain, 28.54% → **23.25–23.37% at k=5–8**, and it is not free: **k=1 by `spread` is worse than not calibrating at all (33.74%)**. A single observation is a noisy estimate of a machine's correction, and applying a bad correction is worse than applying none — which is precisely what the cold-start study found (19.51% at k=1 random against 15.84% uncalibrated). Calibrate with ≥5 queries or not at all.

The residual error is a diagnosed I/O gap

The model still **over-predicts systematically** (mean signed error +0.181; per machine +0.015 to +0.359), and the worst cases are all the same queries:

prod	query	dev	predicted	actual	off by
c8m1g	q2	599.5	602.2	260.9	131%
c4m3g	q2	599.5	604.8	266.3	127%
c4m3g	q20	1405.6	1413.3	656.7	115%

Over-prediction correlates with each query's I/O fraction (Spearman **r = +0.302**). The cause: **the probe is cache-resident and the workload is not.** Probe tables are ~120 MB against a 128 MB buffer pool, so the probe measures compute, memory and parallel scheduling. TPC-H q1 and q6 read 104,000+ blocks — about 850 MB — so they are dominated by the storage path, which differs sharply between a Homebrew cluster on APFS and a Debian cluster on the Docker VM's filesystem.

The probe has no I/O axis. That is the single highest-value change remaining.

Honest scope

- **Same ISA.** All six production cells are containers on the same arm64 host. This crosses an engine-build and storage boundary, which is why it is informative, but it does **not** test the arm64→x86 transfer that broke the original signature at 324%.
- **Single client, identical SF1 data, 22 queries, 6 production machines.** Concurrency remains unmodelled and reaches 8.57× at 8 clients.
- **Probe hygiene is load-bearing.** An earlier pass of this same evaluation, run on probes taken while the machine was busy, reported zero-shot at 47.36% against naive's 41.11% — i.e. the opposite conclusion. Re-probing all cells on an idle machine moved 3 cells from failing the repeatability gate to passing it and reversed the verdict. Any capability probe taken under load should be discarded, and `cetp.cross_env.load_rates` refuses them.

The one-line version

A developer's laptop can predict production query runtime at 28.54% MAPE against 38.22% for assuming production equals the laptop, zero-shot, on 132 pairs over 6 production machines. Measuring 5–8 queries on production takes it to ~23%. The remaining error is a diagnosed, fixable gap: the probe measures no storage throughput.